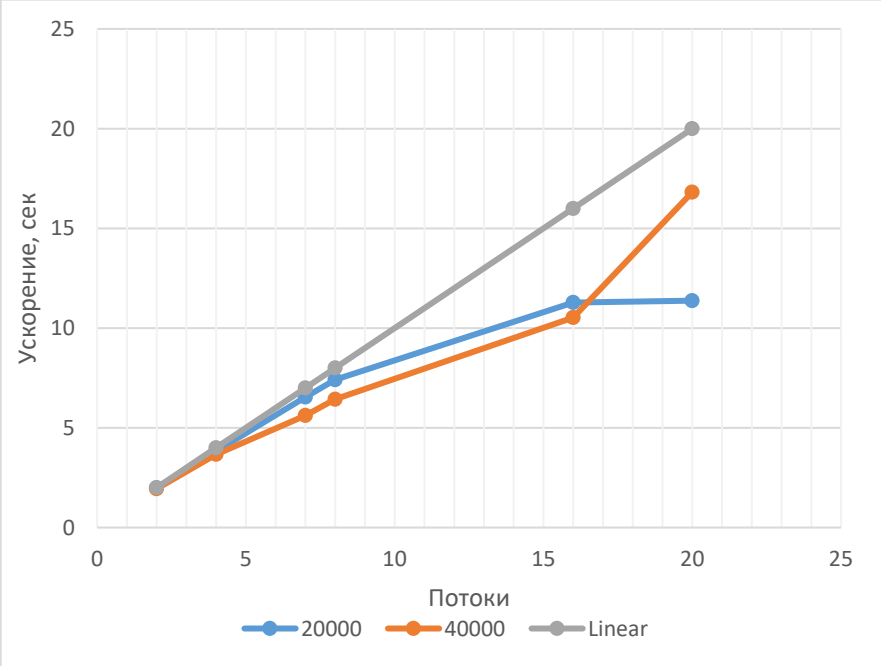
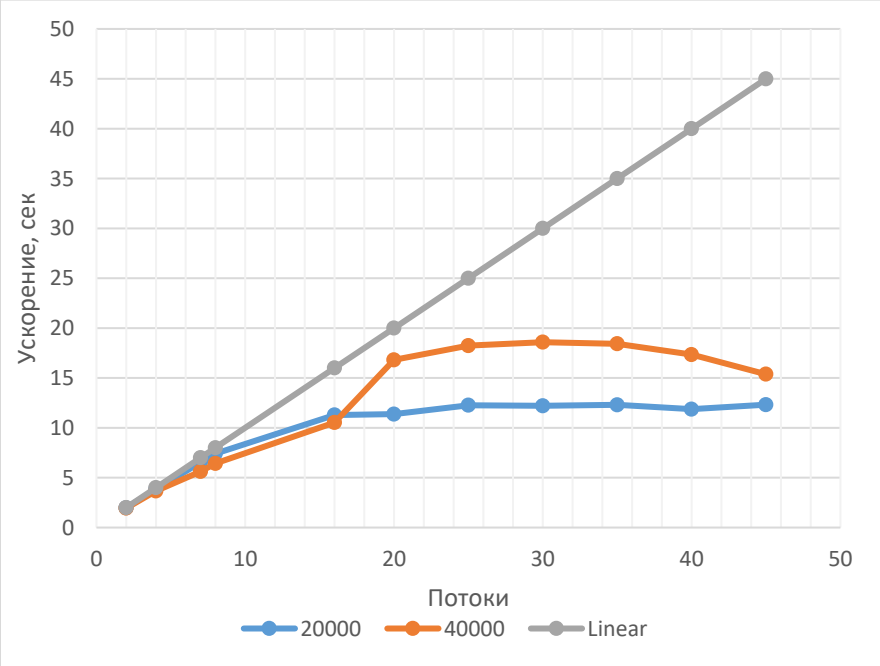
Умножение матрицы на вектор с параллельной инициализацией массивов

Заполнить таблицу для 1,2,4,7,8,16,20,40 потоков. Для элементов массива используйте тип double.

M = N	Количество потоков														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
20000	0.5931	0.2986	1.986	0.1550	3.8258	0.0906	6.5412	0.08	7.4135	0.0525	11.285	0.0521	11.376	0.0499	11.872
40000	2.1836	1.1231	1.9443	0.5953	3.6681	0.3887	5.6173	0.3394	6.4339	0.2074	10.527	0.1298	16.818	0.1259	17.340

Размеры матриц: 20000x20000, 40000x40000. Построить график ускорения в зависимости от количества потоков:





Оптимальное кол-во потоков при 20000: 20, 40000: 35

Вывод о параллельной инициализации массивов:

Так как страницы оперативной памяти выделяются операционной системой в момент первого обращения к ним (записи или чтения). И выделяются они из той памяти, которая физически ближе к тому ядру процессора, которое сделало это первое обращение. Поэтому параллельная инициализация массивов будет предпочтительнее, так как дальнейшее выполнение вычислений будет проходить с памятью, которая уже была загружена в NUMA-узел и не будет происходить подтягивание данных по межсокетной шине.

Вывод о масштабируемости:

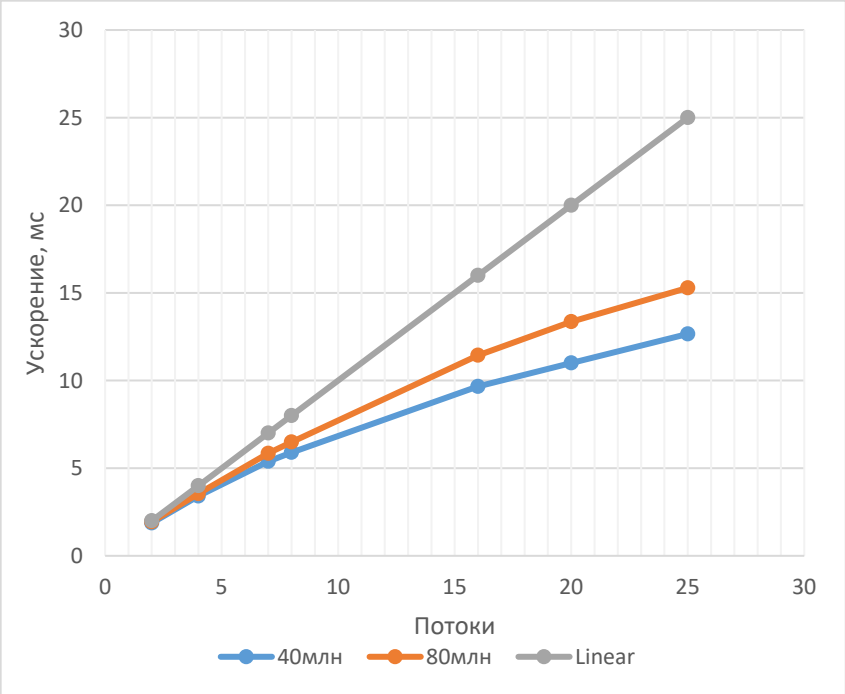
График с 40000 размерностью матрицы дает большее ускорение при увеличении кол-ва используемых потоков, это объясняется тем, что доля полезной вычислительной работы возрастает, а доля накладных расходов на создание потоков, синхронизацию (барьеры) и планирование становится пренебрежимо малой.

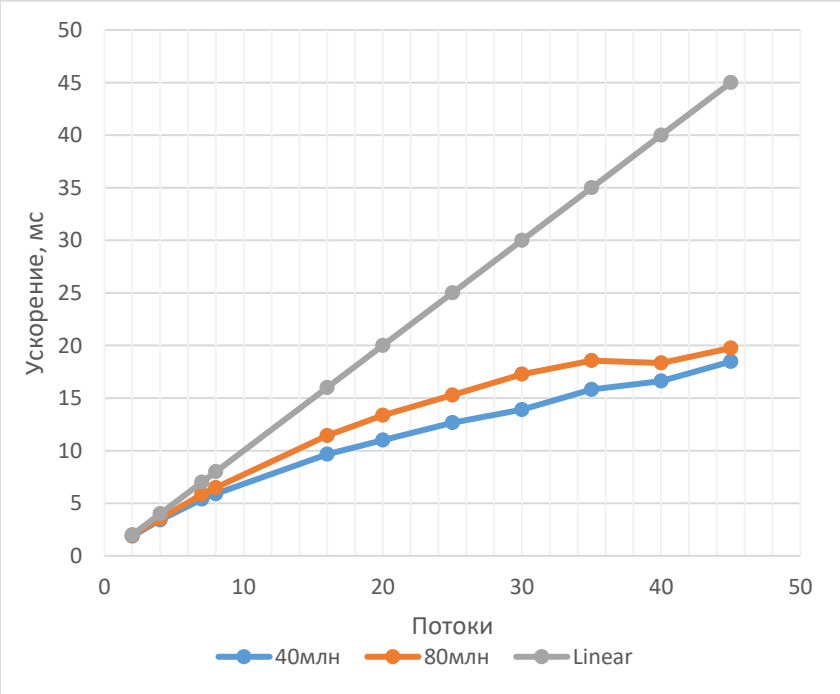
Параллельная версия программы численного интегрирования

Оценить ускорение программы на 1,2,4,7,8,16,20,40 потоках, при числе точек интегрирования nsteps = 40 000 000, 80 000 000

nsteps	Количество потоков														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
40000000	277.67	148.57	1.868	81.471	3.408	51.558	5.3857	47.086	5.8972	28.734	9.6634	25.234	11.004	16.71	16.617
80000000	539.64	278.64	1.9366	152.15	3.5467	92.283	5.8476	83.148	6.49	47.178	11.438	40.404	13.356	29.426	18.338

Построить график ускорения в зависимости от количества потоков.





Оптимальное кол-во потоков при 40млн: 25, 80млн: 35

Вывод о масштабируемости:

Как и в предыдущей задаче мы можем заметить закономерность, что чем больше данных в задача, тем лучше она масштабируема. Это объясняется тем, что Увеличение объема данных приводит к росту доли времени, затрачиваемого на полезные вычисления, в то время как накладные расходы на создание потоков, синхронизацию (барьеры) и редукцию остаются практически неизменными.

Для 40млн данных оптимально использовать 25 потоков, для 80млн – 35, дальнейшее увеличение числа потоков приводит к тому, что накладные расходы и конкуренция за ресурсы кэша начинают преобладать над выгодой от дополнительного распараллеливания. Метрика Эффективности становится меньше установленного порога (0.5).

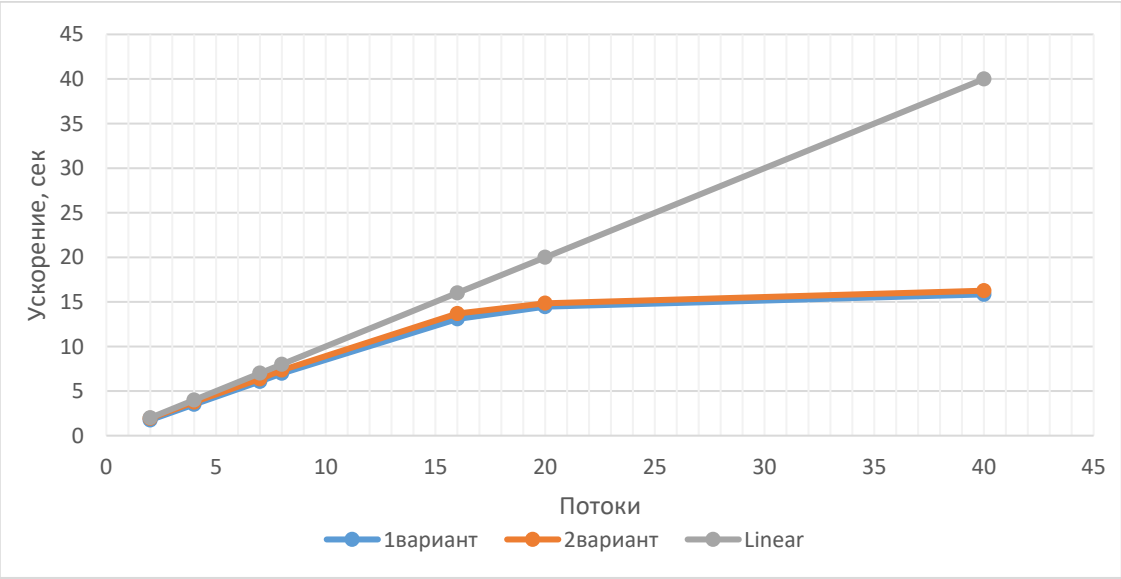
Параллельная реализация решения системы линейных алгебраических уравнений

Вариант 1: для каждого распараллеливаемого цикла создается отдельная параллельная секция #pragma omp parallel for,
Вариант 2: создается одна параллельная секция #pragma omp parallel, охватывающая весь итерационный алгоритм.

Замерить время работы двух вариантов программы при использовании различного числа ядер: от 1 до числа доступных в узле.

вариант	Количество потоков														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
№1	35.658	19.311	1.846	9.920	3.594	5.861	6.084	5.085	7.012	2.727	13.076	2.514	14.184	2.296	15.530
№2	37.325	20.364	1.832	10.193	3.661	5.867	6.361	5.107	7.308	2.724	13.702	2.466	15.135	2.251	16.581

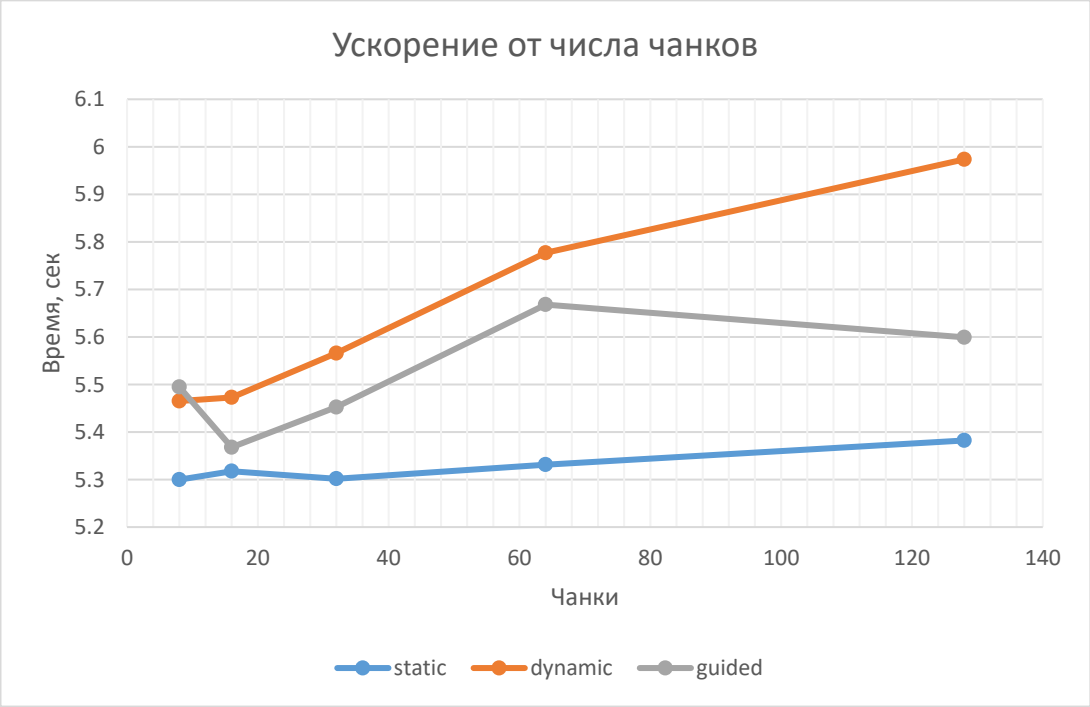
Построить графики зависимости времени работы программы, ускорения и эффективности распараллеливания от числа ядер.



#pragma omp for schedule(static, dynamic, guided, chunk)

Провести исследование на определение оптимальных параметров #pragma omp for schedule(...) при некотором фиксированном размере задачи и количестве потоков.

schedule	chunks	Time, sec
static	8	5.29984
	16	5.31747
	32	5.30173
	64	5.33139
	128	5.38225
dynamic	8	5.46522
	16	5.47277
	32	5.56585
	64	5.77693
	128	5.97356
guided	8	5.49472
	16	5.36797
	32	5.45237
	64	5.66803
	128	5.59915



Вывод:

- static тип оказался стабильнее всех, потому что каждая строка матрицы обрабатывается одинаковое время (это равномерная вычислительная задача)
- dynamic конкурировал на малых размерах чанков с guided, но при их росте время выполнения выросло. Так как один поток может получить последний кусок работы, а остальные ожидают пока он закончит.
- guided показал что-то среднее между static и dynamic он начинает с больших кусков (как static), поэтому работает быстро, но по мере уменьшения остатка задачи он переходит на мелкие куски. В данной задаче это дает небольшой проигрыш по сравнению с чистой статикой, но защищает от сильного падения производительности на больших чанках, которое у наблюдается dynamic.

Вывод о целесообразности использования первого или второго варианта программы:

Динамические системы вносят избыточные накладные расходы и чувствительны к выбору размера чанка: слишком большой чанк приводит к дисбалансу загрузки потоков и росту времени выполнения. Второй вариант использования целесообразней.